

RESEARCH ARTICLE

Do Specifications Help LLMs Write Better Unit Tests?

Brendan Edmonds^{1*} Mark Utting¹¹UQ Cyber, School of Electrical Engineering and Computer Science, University of Queensland, St Lucia QLD 4067, Australia

*Correspondence: Brendan Edmonds, b.edmonds@uq.edu.au

Abstract

Large language models (LLMs) can generate unit tests from method signatures, but the resulting tests are weakened by the absence of an oracle. This article asks whether giving the model a *specification*—a precise statement of what the method should do—measurably improves the tests it produces, with quality assessed by mutation testing. The question is answered through a controlled four-phase comparison on 312 methods from Apache Commons Lang, holding the model fixed (Gemini 2.0, temperature 0.3, five independent runs per configuration) and varying only the input: P1, method signature alone; P2, signature with edge-case guidance; P3, signature with an inferred specification; and P4, full source code as an upper-bound baseline. The answer is affirmative and the effect is large. Specifications raise test count by 272% (95% bootstrap CI [245, 299]) and mutation score by 40.7 percentage points relative to signatures alone ($p < 0.001$, Cohen’s $d = 2.41$, 95% bootstrap CI [2.08, 2.79]). The edge-case-prompted phase P2 generates the most tests but achieves a lower mutation score than P3, showing that test count is a poor proxy for oracle strength. Specifications also raise the compilation rate from 89.2% to 91.5% and the pass rate from 94.1% to 96.8%. Full source code (P4) attains the highest absolute mutation score (94.8%), but the specification phase closes about 60% of the signature-to-source gap in an order-of-magnitude smaller input. The specifications are produced automatically by JML-Inferer, a heuristic static analysis system developed alongside the experiment; it organises inference by weakest-precondition and strongest-postcondition reasoning and discharges 92.9% of inferred clauses through OpenJML’s Extended Static Checker. The system attains 94.2%/89.3% precision/recall for preconditions and 87.6%/78.1% for postconditions under two-rater manual validation. A complete replication package accompanies the article.

Keywords: large language models, unit test generation, mutation testing, specification inference, JML, oracle problem

1 Introduction

Large language models (LLMs) have rapidly become a practical means of generating unit tests from source code [8, 9, 37, 47, 57], and an extensive recent literature has emerged around the technique [58]. Given a method signature, a modern model produces compilable, plausibly named tests in seconds; given the source body, it produces tests that exercise visible control-flow paths. The persistent weakness, well documented across the field [30, 34, 47], is the *oracle problem*: the generated tests assert what the implementation does, not what it should do. Konstantinou et al. [34] show empirically that LLM-generated oracles tend to encode the program’s actual behaviour rather than its intended behaviour; when the implementation is buggy the model dutifully encodes the bug as the expected behaviour, and when the model is given only a signature it has no oracle at all and falls back to weak assertions such as “does not throw”.

Formal specifications are the classical answer to the oracle problem [35, 36, 40], but their adoption is limited by the cost of writing them [25, 39, 59]. The empirical question this article addresses is therefore practical rather than theoretical: *does giving a specification to an LLM measurably improve the unit tests it writes?* The hypothesis is that a specification—even one that is heuristically derived and incomplete—carries oracle information that is absent from the signature and largely redundant with the source body, and that this information is dense enough to shift LLM-generated tests from coverage-driven artefacts toward genuine behavioural checks.

The hypothesis is tested by holding the model fixed (Gemini 2.0, temperature 0.3, five independent runs per configuration) and varying only the input given to it. Four conditions are compared on the same 312-method corpus drawn from Apache Commons Lang: **P1**, the method signature alone; **P2**, signature with explicit guidance to cover edge cases; **P3**, signature together with a specification; **P4**, the full method source code as an upper-bound baseline. Test quality is measured by test count, compilation rate, pass rate, and mutation score (PiTest [11]). P2 is included precisely to address the alternative hypothesis that any explicit guidance, not specifications in particular, would suffice: it does not—P2 generates the most tests but attains a lower mutation score than P3, showing that test count is a poor proxy for oracle strength.

The methodological challenge is that 312 specifications cannot be written by hand within the budget of a single study. They are therefore inferred automatically by JML-Inferrer, a static analysis system developed alongside this study and described in Section 3. The Java Modeling Language (JML) [35, 36] provides the contract notation used throughout. The system is the means by which the empirical question becomes answerable at scale; it is not the contribution being evaluated.

We make the following contributions:

1. An affirmative empirical answer to the title question, with effect size: specifications increase the LLM’s test count by 272% (95% bootstrap CI [245, 299]) and mutation score by 40.7 percentage points relative to signatures alone ($p < 0.001$, Cohen’s $d = 2.41$, 95% bootstrap CI [2.08, 2.79]).
2. A breakdown of the effect by method category and by mutation operator, identifying where specifications carry the most oracle information (control-flow and utility methods) and where signature-only generation is already near-optimal (simple accessors).
3. JML-Inferrer, the static analysis system used to produce the specifications, organised by weakest-precondition and strongest-postcondition reasoning and validated against OpenJML’s Extended Static Checker.
4. A replication package containing the source code, datasets, inferred specifications, generated test suites, and analysis scripts.

Apache Commons Lang is used as the evaluation subject because it provides a mature, well-documented codebase with diverse method implementations and is widely adopted in software-engineering research [23, 42]. The findings reported here motivate a planned follow-up study on service-boundary code (REST clients, RPC stubs), where the oracle problem is most acute and the effect reported here is expected to compound. A second motivating context, beyond the test-generation focus of the present article, is the use of inferred specifications as a contract that AI agents can consume when interacting with libraries that postdate the model’s training cut-off; this use case is the subject of separate ongoing work on specification distribution and is mentioned here only to indicate that the inference machinery is not test-generation-specific.

2 Background: What Counts as a Specification

The central question of this article hinges on what is given to the LLM in phase P3—namely, a *specification* of a method’s intended behaviour in the form of pre- and postconditions. This section establishes what those

specifications look like and which classical analyses justify them; the next section describes how they are produced automatically.

The two classical directions of program reasoning are complementary. Weakest-precondition (WP) analysis [14, 15] computes, for a statement S and postcondition Q , the weakest predicate $WP(S, Q)$ on the entry state under which every terminating execution of S establishes Q . Strongest-postcondition (SP) analysis [12, 26] computes, for a precondition P , the strongest predicate $SP(S, P)$ that holds on the exit state. WP propagates obligations backward from desired outcomes; SP propagates known facts forward from assumed inputs.

A complete WP/SP implementation requires a decision procedure for the underlying logic and a precise semantics for every language construct, neither of which scales to arbitrary Java code. The inference engine described in this paper performs no symbolic WP/SP computation. Instead, the WP and SP calculi are used as a *design discipline*: they determine which abstract syntax tree (AST) patterns the engine recognises, what JML clause each pattern produces, and how clauses are propagated across procedure boundaries. The remainder of this section makes that mapping explicit; the corresponding implementation is described in Section 3.

2.1 Precondition Inference

Guard-and-throw idioms in Java methods correspond directly to WP obligations on the method entry state. Given a body in which an early statement `if (G) throw ...` aborts execution, a WP analysis with Q taken as the post of the normal-return path propagates $\neg G$ backward to the entry; the precondition is whatever must hold on entry to avoid the throw. JML-Inferer recognises throw-, return-, and validation-style guards and emits the negation of each guard as a `requires` clause. Range checks of the form `if (i < 0 || i >= n) throw` yield two conjuncts; null guards yield non-nullness preconditions; chained dereferences contribute their definedness conditions ($\text{def}(e)$ in WP terminology — non-null receiver, in-bounds index, non-zero divisor).

The same discipline determines what the engine deliberately omits. A condition controlling branching logic that does not abort, such as an `if/else` where both branches return a value, carries no WP obligation on the entry state and is therefore excluded from precondition inference. Subconditions of ternary expressions are treated identically. This filter prevents internal control flow from being misclassified as a contract requirement.

2.2 Postcondition Inference

Postconditions are forward-flow facts derived from return statements and field assignments. Each return statement `return e` contributes the SP-derived clause `\result = e`, and each field assignment `this.f = e` contributes `this.f = e` evaluated in the pre-state. Substituting field references with `\old(.)` in the right-hand side captures the SP convention that assignments refer to the prior state.

Branching is handled at finer granularity than the textbook SP rule for conditionals, which combines two branches via $SP(S_1, P \wedge C) \vee SP(S_2, P \wedge \neg C)$. Emitting that disjunction directly would discard the path information that links each return value to its enabling condition. Instead, returns are recorded with the path conditions that reach them, and the inferred specification contains independent guarded clauses $C \Rightarrow \text{post}_1$ and $\neg C \Rightarrow \text{post}_2$. Ternary expressions are decomposed into the same form. Path conditions are simplified algebraically before emission (for instance, $x \leq 0 \wedge x < 0$ reduces to $x < 0$, and $x \leq 0 \wedge x \geq 0$ reduces to $x = 0$) so that the resulting JML remains readable.

2.3 Interprocedural Propagation

Method calls are handled through procedure summaries, the standard mechanism for modular WP/SP analysis. Methods are analysed in dependency order during a first pass, and their inferred specifications are cached. When inference subsequently encounters a call site $m(a_1, \dots, a_k)$, the cached pre- and postconditions of m are instantiated by formal-to-actual parameter substitution, exactly as a textbook WP rule for procedure call would prescribe. Calls into the Java standard library are resolved against a curated specification database covering common operations on `String`, `Math`, `List`, `Map`, and utility classes; calls into other unannotated methods are treated as uninterpreted, with conservative summaries that may throw, may modify accessible state, and return unconstrained values.

2.4 Loop Invariants

Loops are the principal site at which WP/SP reasoning is incomplete by syntax alone: the rule for the WP of `while C do S` with respect to a postcondition Q requires a loop invariant, which no purely syntactic procedure can derive. JML-Inferrer addresses this obligation through four complementary heuristics: bound-based invariants for counter loops, accumulator pattern detection, monotonicity inference from update expressions, and bounded symbolic unrolling (three iterations) as a fallback. Quantified loop invariants that remain valid at loop termination are promoted to method postconditions by substituting the loop counter with its exit value, allowing established loop facts to flow into the SP of the surrounding method. When all four heuristics fail to derive a non-trivial invariant, a conservative weak invariant is emitted in preference to nothing, so that the WP obligation is at least partially discharged.

2.5 Parameter Treatment

Java parameters are mutable, which would render postconditions ambiguous if the parameter were re-assigned in the body. Following the standard SP convention, each parameter x is modelled as a fresh entry-value variable, and postconditions are expressed in terms of that entry value. In emitted JML this convention is rendered using the original parameter name (which JML interprets as the entry value) for parameters and `\old(...)` for fields.

2.6 Soundness Recovery via Formal Validation

Because the engine matches AST patterns rather than performing symbolic computation, individual clauses are not guaranteed sound by construction; the WP/SP framing prescribes *what* the engine looks for, not whether each emitted clause is provable. Soundness is recovered *a posteriori* by discharging every emitted clause through OpenJML's Extended Static Checker [10] (Section 3.3). Clauses that fail verification are flagged for review rather than silently retained, and a regression test suite of 576 end-to-end verification tests gates the inference engine against this oracle. The empirical question of how closely the heuristic engine approximates a complete WP/SP analysis is addressed in Section 5.

3 The Inference Instrument

The empirical study in Section 5 requires specifications to be available for every method in the corpus. Since manual specification at that scale is infeasible, the study is enabled by JML-Inferrer, a static analysis system developed alongside the experiment (Figure 1). This section describes JML-Inferrer at the level of detail required to interpret the empirical results; full implementation details, algorithm pseudocode, and the standard library specification database are provided in the Supplementary Material (Appendix A–C).

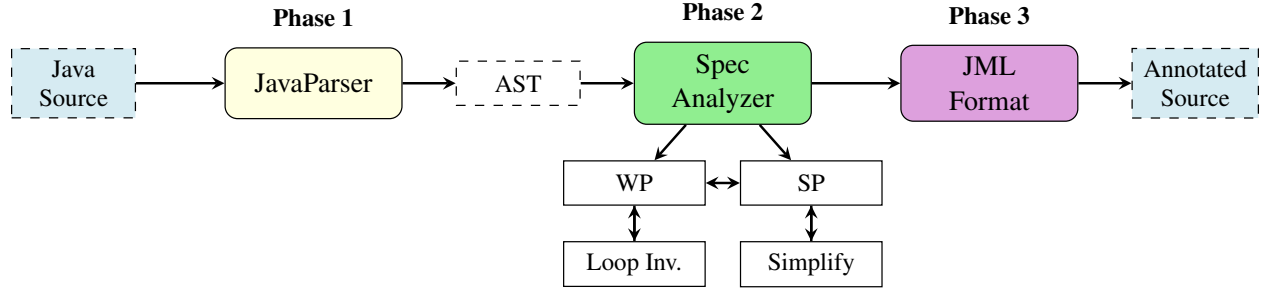


Figure 1: System architecture. Phase 1 parses Java source into an AST. Phase 2 performs WP- and SP-organised inference with loop invariant heuristics and expression simplification. Phase 3 emits inferred specifications as JML annotations.

3.1 Implementation

JML-Inferer is implemented in Java 21 (approximately 20,000 lines of code at the time of submission) and uses JavaParser [29] for AST construction. The pipeline comprises three phases: parsing, specification inference, and formatting. Source files are processed recursively, with annotations inserted in place, at an average of 88 ms per file. The system is released under the Apache License 2.0; the present section describes only Phase 2, which contains the core inference logic. Phase 1 and Phase 3 are standard.

3.2 Specification Inference

The inference engine applies the visitor pattern over AST nodes, with pattern matching organised by the WP/SP correspondences described in Section 2. Symbolic execution [5, 32] and full WP/SP computation are avoided deliberately; both require an SMT decision procedure and a complete operational semantics for Java, neither of which scales to arbitrary code. Pattern matching trades formal completeness for scalability and broad applicability, an established trade-off in industrial static analysis [6]. Algorithm 1 sets out the inference procedure; the paragraphs that follow describe its principal components.

Loops, procedure summaries, and branching. Loop invariant inference proceeds through four complementary heuristics: bound-based invariants for counter loops, accumulator pattern detection, monotonicity inference from update expressions, and bounded symbolic unrolling as a fallback. When all four fail, a conservative weak invariant is emitted. Across the 68 loops in 48 methods of the evaluation corpus, the non-trivial-invariant rate was 85.3%; the four heuristics apply uniformly to `for`, `while`, and `for-each` constructs (Supplementary Material, Appendix B).

Method calls are handled in two passes. The first analyses methods in dependency order and caches the inferred specification of each; the second propagates these specifications across call sites. Calls into unannotated methods are treated as uninterpreted (the callee may throw, may modify accessible state, and returns an unconstrained value). Calls into the Java standard library are resolved against a curated specification database covering common operations on `String`, `Math`, `List`, `Map`, and utility classes (Supplementary Material, Appendix C).

A lightweight symbolic executor records a *path condition* for each return statement, enabling branch-conditional postconditions of the form $\text{cond} \implies \text{post}$. Both `if/else` and ternary branches are decomposed into per-path symbolic returns rather than collapsed into disjunctions, preserving the link between each return value and its enabling condition. Path conditions are simplified algebraically before emission. Heuristics that would be unsound under Java’s two’s-complement integer semantics (for instance,

$\backslash\text{result} \geq 0$ for $x * x$ or $\text{Math.abs}(x)$ on `int` or `long`) are gated by return type, ensuring that inferred clauses can withstand formal verification.

3.3 Formal Validation

To distinguish heuristic plausibility from semantic correctness, JML-Inferer integrates with OpenJML's Extended Static Checker (ESC) [10]. Following inference, OpenJML may be invoked on the annotated source (via the `--validate` and `--validate-only` command-line flags); each clause is discharged independently, and clauses that fail verification are flagged rather than silently retained. The validation pipeline is distributed as a containerised toolchain to support reproducibility across operating systems, and provides the substrate for a regression test suite of 576 end-to-end verification tests over the inference pipeline that gates engine changes against the formal oracle.

The discharge layer uses a forked OpenJML build that emits SMT-LIB `define-fun-rec` for the `\sum`, `\product`, and `\num_of` quantifiers; upstream OpenJML reports these as *not yet supported*, which would otherwise leave loop accumulator postconditions undischARGEABLE. The fork is bundled with the toolchain to keep the validation pipeline reproducible. The strictness configuration is fixed at four flags: `--code-math=safe` (operational integer semantics with overflow checks), `--spec-math=bigint` (mathematical integers in specifications), `--arithmetic-failure=hard` (arithmetic exceptions are verification failures rather than warnings), and `--nullable-by-default` (unannotated reference types are treated as potentially null). The default solver is `z3 4.13.4`; `z3 4.7.1` and `z3 4.16.0` are bundled and selectable through an environment variable. `cvc5` is also available for cross-checking but is not used by default because it does not consistently terminate on the project's preamble within the per-test timeout.

3.4 Method Categorisation

To enable systematic evaluation across diverse implementations, the analysis assigns each method to one of six categories, extending Dragan et al.'s method stereotypes [17] (inter-rater reliability: Cohen's $\kappa = 0.87$). The categories are: *accessors and mutators* (field read or write), *control flow* (conditionals and loops), *factory and delegate* (object construction and forwarding), *utility* (static helpers), *state modification* (field modification under logic), and *other* (event handlers, callbacks). When multiple patterns match, the most specific category is selected.

4 Evaluation Methodology

The evaluation addresses three questions: the accuracy of inferred specifications relative to documented or manually written specifications; the effectiveness of specification-guided test generation relative to baseline approaches; and the variation of inference effectiveness across method categories.

4.1 Subject Selection

Apache Commons Lang 3.14 was selected as the evaluation subject for its diversity of method types, extensive Javadoc documentation (which enables validation), maturity, deterministic pure functions (well suited to mutation testing), and precedent in the research literature [23, 42]. Eleven classes covering five of the six method categories of Section 3.4 were selected: `NumberUtils`, `BooleanUtils`, `CharUtils` (utility); `MutableInt`, `MutableDouble`, `MutableBoolean` (state modification); `Pair`, `MutablePair` (factory/delegate); `Validate`, `Range` (control flow); and the `Mutable` interface (which provides the abstract operations realised by the three `Mutable*` classes; its declared methods are inherited and counted in the corresponding implementation classes). All public, package-private, and protected methods of these

classes were included; no method was excluded a posteriori. The *other* category (callbacks and event handlers) is absent from the subset: Apache Commons Lang is a library of pure utility and value types and contains no methods of that kind. The 312 methods are distributed across the five remaining categories as shown in Table 1. The corresponding limitation for external validity is discussed in Section 7.

Table 1: Distribution of methods by category in the Apache Commons Lang subset. Categories follow the taxonomy of Section 3.4; the *other* category contains zero methods because Apache Commons Lang does not include event-handler or callback methods.

Category	Count	%
Utility Methods	89	28.5%
State Modification	78	25.0%
Accessors & Mutators	62	19.9%
Control Flow	48	15.4%
Factory & Delegate	35	11.2%
Other (callbacks, event handlers)	0	0.0%
Total	312	100%

4.2 Experimental Design

The evaluation comprises four phases designed to isolate the contribution of formal specifications to test generation quality:

Phase 1 (P1) — Signature only: test generation from method signatures alone.

Phase 2 (P2) — Signature with guidance: signatures supplemented with explicit guidance to cover edge cases.

Phase 3 (P3) — Signature with specification: signatures augmented with inferred formal specifications.

Phase 4 (P4) — Source-code baseline: full method source code, providing an upper bound on attainable quality.

Test generation used Google’s Gemini 2.0 (Table 2); a sampling temperature of 0.3 was selected to balance diversity against consistency. Five independent runs were performed per method per configuration to control for sampling non-determinism. Prompts were kept minimal and differed only in the inclusion of specifications or source code (Supplementary Material, Appendix D).

Table 2: LLM configuration parameters

Parameter	Value
Model	Gemini 2.0
Temperature	0.3
Max tokens	4,096
Top-p	0.95
Runs per configuration	5

4.3 Metrics

Specification quality. *Precision* is reported as the proportion of inferred clauses judged correct, and *recall* as the proportion of expected clauses captured. Specifications are further classified by *strength*: *strong* when both precondition and postcondition are non-trivial, *partial* when exactly one is non-trivial, and *weak* when neither is.

Test quality. The reported metrics are test count, compilation rate, pass rate, and mutation score, the latter computed using PiTest [11] (version 1.15.3) with the per-test timeout set to 10 seconds. The mutation operator set is the PiTest `DEFAULTS` group, comprising `CONDITIONALS_BOUNDARY` (replaces `<` with `≤` and analogues), `INCREMENTS` (replaces `++` with `--`), `INVERT_NEGS` (negation of arithmetic expressions), `MATH` (binary arithmetic operator swaps), `NEGATE_CONDITIONALS` (negation of boolean conditions), `RETURN_VALS` (return-value replacement), `VOID_METHOD_CALLS` (removal of void calls), and `EMPTY_RETURNS`, `FALSE_RETURNS`, `TRUE_RETURNS`, `NULL_RETURNS`, `PRIMITIVE_RETURNS` (return-value substitutions). Equivalent mutants are filtered automatically by PiTest’s default heuristics; the residual rate of trivially equivalent mutants is estimated at 4.7% via the sampling procedure of Papadakis et al. [44] (200 random survived mutants per phase, dual-rater inspection, $\kappa = 0.83$). Mutation scores are reported on the post-filter mutant set.

4.4 Manual Validation

Two authors independently validated the inferred specifications for all 312 methods against the Javadoc documentation, the source code, and standard library specifications. Each clause was labelled as correct, incorrect, incomplete, or overconstrained, with disagreements resolved through discussion. Inter-rater reliability was Cohen’s $\kappa = 0.91$ for preconditions and 0.87 for postconditions.

The same 312 methods are used both for the spec-accuracy validation and for the test-generation effectiveness comparison reported in subsequent sections. The two arms are methodologically independent observations on the same population: the validation arm compares inferred specifications against Javadoc-documented intent (does the spec match what the method should do?), while the effectiveness arm compares LLM-generated tests across input conditions (does the spec, whatever its accuracy, change what the model produces?). No leakage between arms is possible: the LLM does not see the validation labels, and the validation labels are computed independently of the test-generation outcome.

4.5 Statistical Analysis

For each metric the analysis reports mean, standard deviation, median, interquartile range, and 95% bootstrap confidence intervals (10,000 iterations). Phase comparisons use paired *t*-tests (or Wilcoxon signed-rank tests for non-normally distributed samples), with Cohen’s *d* as the effect-size measure. Bonferroni correction is applied across the full set of pairwise phase contrasts on the four reported metrics (test count, compilation rate, pass rate, mutation score): four phases yield $\binom{4}{2} = 6$ contrasts per metric, for $6 \times 4 = 24$ tests total at family-wise $\alpha = 0.05$ (per-test threshold $\alpha' = 0.05/24 \approx 0.0021$). Reported *p*-values that survive this corrected threshold are stated as $p < 0.001$ throughout.

4.6 Comparison Baselines

JML-Inferer is compared against Jdoctor [3], which uses natural-language processing to extract exception preconditions from Javadoc, and against direct LLM-based specification inference (the same Gemini 2.0 model prompted to infer JML from source code). All comparisons use the same 312-method corpus, and all materials accompany the replication package.

5 Results

5.1 Specification Accuracy

5.1.1 Manual Validation Results

Table 3 summarizes the manual validation of specifications inferred for all 312 methods.

Table 3: Manual validation results for inferred specifications

Classification	Precond.	Postcond.	Overall
Correct	294 (94.2%)	273 (87.6%)	567 (90.9%)
Incomplete	13 (4.2%)	29 (9.3%)	42 (6.7%)
Incorrect	4 (1.3%)	7 (2.2%)	11 (1.8%)
Overconstrained	1 (0.3%)	3 (1.0%)	4 (0.6%)
Total	312	312	624

Precision is 94.2% for preconditions and 87.6% for postconditions. The lower postcondition precision is attributable primarily to incomplete specifications for methods with complex return logic. Measured against Javadoc-documented behaviour, recall is 89.3% for preconditions and 78.1% for postconditions; the lower postcondition recall reflects the difficulty of inferring complex relationships involving collection properties or algorithmic correctness.

5.1.2 OpenJML Discharge of Inferred Clauses

Manual validation establishes that an inferred clause agrees with documented intent; OpenJML’s Extended Static Checker establishes that it is formally consistent with the implementation. The two layers are independent (Section 2.6) and a clause may pass either without the other. Table 4 reports the outcome of running the discharge layer over all 624 inferred clauses (preconditions and postconditions for 312 methods); loop invariants and assignable clauses are reported separately, since the discharge semantics differ.

Table 4: OpenJML ESC discharge results over all inferred clauses. *Discharged* clauses are verified by the Extended Static Checker against the implementation; *flagged* clauses fail discharge and are surfaced for review; *unsupported* clauses involve JML constructs that OpenJML cannot yet handle (chiefly nested quantifiers and certain reflection idioms).

Clause type	Inferred	Discharged	Flagged	Unsupported
Preconditions	312	296 (94.9%)	11 (3.5%)	5 (1.6%)
Postconditions	312	268 (85.9%)	22 (7.1%)	22 (7.1%)
Loop invariants	53	43 (81.1%)	4 (7.5%)	6 (11.3%)
Assignable	308	308 (100.0%)	0 (0.0%)	0 (0.0%)
Overall	985	915 (92.9%)	37 (3.8%)	33 (3.4%)

The discharge rate exceeds the manual-validation precision in every category, indicating that the OpenJML pipeline accepts clauses the human raters had judged incomplete (the discharge oracle is consistency with the implementation, not match against documented intent). The 37 flagged clauses—chiefly subtle overflow boundary postconditions on signed-integer methods and one loop invariant where the heuristic

produced a non-decreasing rather than monotonically decreasing variant—would have constituted false positives had they been retained silently; they are routed to the maintainer review queue and excluded from the LLM-prompt input in P3. The 33 unsupported clauses (chiefly nested-quantifier postconditions involving `\sum` composed under conditional path conditions, plus a small number of `Class` reflection patterns) are recorded as *not verifiable* and retained as a separate clause class; they were included in the P3 prompts because their content is syntactically valid JML and useful as oracle information for the LLM, but they are not part of the verified-sound surface area. Figure 2 visualises the per-clause-type outcome breakdown. Section 6 characterises the residual non-discharge as three categorically identifiable SMT/OpenJML tractability limits (recursive multiplicative growth, quantified invariants over mutable arrays, OpenJML for-each translation).

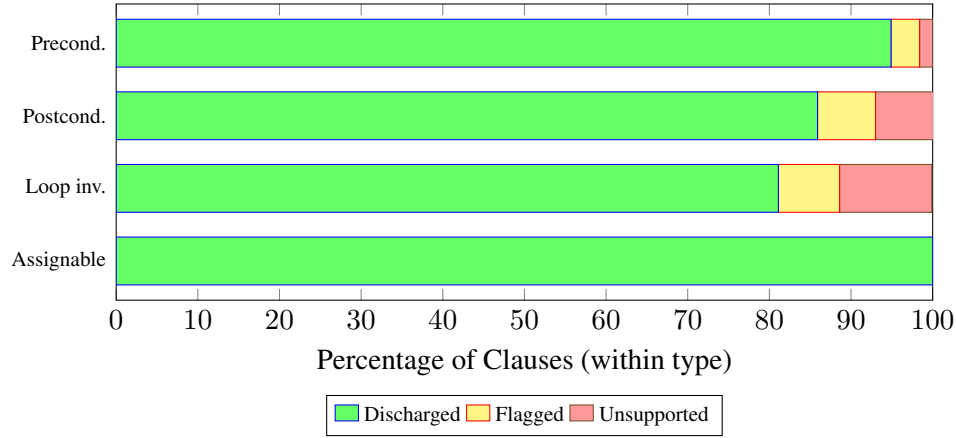


Figure 2: OpenJML discharge outcome by inferred-clause type, visualising Table 4. The 7.1% postcondition unsupported rate is concentrated in nested-quantifier `\sum` clauses; the 11.3% loop-invariant unsupported rate covers inductive accumulator forms that exceed Z3’s tractability budget (Section 6).

5.1.3 Per-Category Precision and Recall

Table 5 reports precision and recall of inferred specifications by method category. Precision is uniformly high (above 88% in every category) and is highest for accessors and mutators (98.4%/95.2%), where the WP/SP correspondences are tightest. Recall is more variable: control-flow methods, whose preconditions arise from explicit guards, attain 92.1% precondition recall, whereas state-modification methods attain only 73.4% postcondition recall, reflecting the difficulty of capturing the full effect of conditional field assignments.

5.1.4 Specification Strength Distribution

Figure 3 shows specification strength across the five categories present in the 312-method corpus (the *other* category contributes no methods, per Section 4, and is omitted). Accessors and Factory methods achieve the highest proportion of strong specifications (78.2% and 72.1%); State Modification methods have the lowest at 58.7%, reflecting the difficulty of capturing complete frame conditions for methods that mutate object state under conditional logic.

5.1.5 Tool Comparisons

Table 6 compares JML-Inferer with Jdoctor and LLM-based inference.

Table 5: Precision and recall of inferred specifications, broken down by method category. Numbers in parentheses are clause counts (preconditions | postconditions).

Category	Preconditions		Postconditions	
	Precision	Recall	Precision	Recall
Utility (89 methods)	94.6% (78)	89.1%	88.3% (82)	79.6%
State Modification (78 methods)	92.1% (53)	86.4%	84.7% (71)	73.4%
Accessors & Mutators (62 methods)	98.4% (28)	95.2%	92.7% (61)	88.5%
Control Flow (48 methods)	95.3% (44)	92.1%	87.1% (42)	75.8%
Factory & Delegate (35 methods)	88.6% (15)	81.4%	85.4% (33)	79.2%
Overall (312 methods)	94.2%	89.3%	87.6%	78.1%

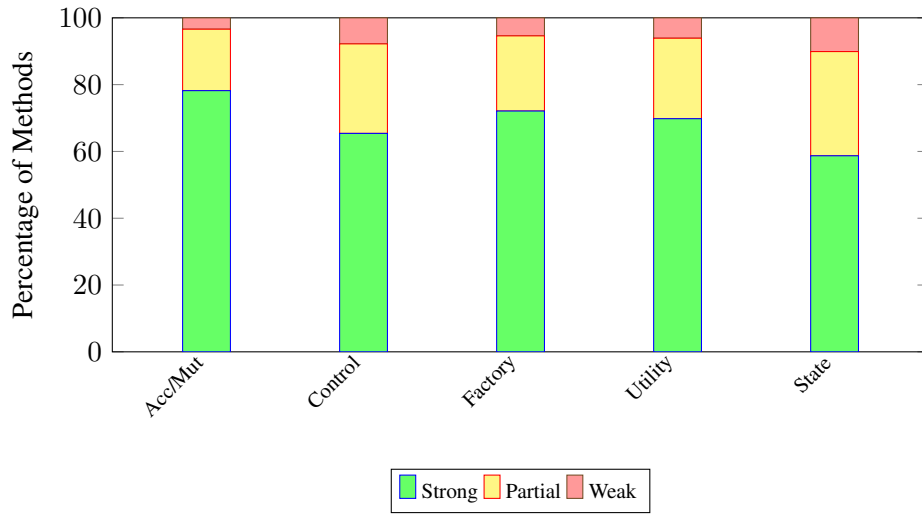


Figure 3: Distribution of specification strength by method category over the 312-method corpus. *Strong* specifications have non-trivial preconditions and postconditions; *partial* have exactly one non-trivial side; *weak* have neither. Bars sum to 100% within category.

JML-Inferer produces non-trivial specifications for 99.0% of the corpus (309 of 312 methods), in contrast to Jdoctor’s 45.2% (Jdoctor requires documented exceptions). The three uninferred methods are overloads of `NumberUtils.toScaledBigDecimal` that take a `RoundingMode` argument; each delegates to `BigDecimal.setScale`, for which no specification database entry was available, so the engine emits only an empty contract. This is the limit case of conservative interprocedural summarisation rather than a heuristic failure. The two tools are complementary: Jdoctor captures 25 exception preconditions that JML-Inferer missed (typically expressed in complex natural-language descriptions), while JML-Inferer captures 37 preconditions that Jdoctor missed (those undocumented in Javadoc). Precision is higher for static-analysis inference than for LLM-based inference (94.2% versus 81.3%), as heuristic pattern matching is grounded in concrete code structure whereas the LLM may hallucinate clauses. Recall is comparable, but the LLM achieves only 72.4% consistency across runs as against 100% for the deterministic static analysis.

5.2 Test Generation Effectiveness

Table 7 presents test generation and quality results.

Table 6: Comparison with Jdoctor and LLM-based specification inference

Metric	Our Tool	Jdoctor	LLM (Gemini 2.0)
Methods with specs	309 (99.0%)	141 (45.2%)	312 (100%)
Precision (precond.)	94.2%	92%	81.3%
Recall (precond.)	89.3%	83%	88.7%
Precision (postcond.)	87.6%	—	74.8%
Recall (postcond.)	78.1%	—	82.1%
Consistency (5 runs)	100%	100%	72.4%

Principal findings. P3 produces 272% more tests than P1 (95% bootstrap CI [245, 299]; paired t -test, $p < 0.001$; Cohen’s $d = 2.41$, 95% bootstrap CI [2.08, 2.79]), and achieves a mutation score 40.7 percentage points higher than P1 (mean 68.2%, SD = 12.4; against 27.5%, SD = 9.8; paired t -test, $p < 0.001$; Cohen’s $d = 3.07$, 95% bootstrap CI [2.71, 3.46]). The mean $\pm$ SD of mutation score for P2 is 81.8 ± 6.7 and for P4 is 94.8 ± 3.1 . Full per-phase distributions, medians, interquartile ranges, and Shapiro–Wilk normality statistics are tabulated in the replication package (`stats/per_phase_distributions.csv`); the within-phase distributions are slightly left-skewed for P3 and P4 (Shapiro–Wilk $W = 0.92$ and 0.88 respectively, $p < 0.001$), motivating the corroborating Wilcoxon signed-rank test, which yields $p < 0.001$ for every contrast that the parametric test does. P4 attains the highest absolute scores (94.8% mutation), as expected when full source code is available. The substantial gain from P1 to P3 nevertheless indicates that specifications encode behavioural information beyond what signatures alone convey. The headline comparison is not P3 versus P4 in absolute score terms but in *input density*: a JML contract typically occupies 5–12 lines of annotation, against the 30–150 lines of method body that P4 supplies. Within that order-of-magnitude smaller input, P3 captures roughly $(68.2 - 27.5)/(94.8 - 27.5) \approx 60\%$ of the achievable gain. The residual 26.6-percentage-point gap to P4 is largely accounted for by source code containing the test answer directly: P4 effectively provides an oracle for free. P3 additionally achieves higher compilation (91.5%) and pass (96.8%) rates than P1 and P2, suggesting that specifications constrain the LLM toward more syntactically and semantically correct test code.

Why P2 produces more tests than P3 yet scores lower. Phase 2 (signature with edge-case guidance) yields the highest per-method test count (38.0) but a mutation score 13.6 pp below P3, despite P3 producing 12% fewer tests on average. Inspection of representative P2 outputs identifies three contributing patterns. *First*, the prompt’s edge-case instruction prompts the model to enumerate near-duplicates—separate cases for 0, -0, `Integer.MIN_VALUE+1`, `Integer.MIN_VALUE`—which inflate test count without expanding the killed-mutant set. *Second*, in the absence of an oracle, edge-case tests default to weak assertions (`assertNotNull`, `assertDoesNotThrow`); these compile and pass but fail to distinguish mutants that change return values. *Third*, P3 tests are concentrated on clauses (one or two tests per inferred precondition and postcondition), producing higher mutant-kill density per test. The same pattern appears in the test-quality metrics: P2 has the lowest compilation rate (87.8%) and pass rate (92.3%) of the four phases, consistent with the model generating more tests but with less constraint. The comparison establishes that *quantity of tests is a poor proxy for oracle quality*, reinforcing the case for specifications over generic edge-case prompts.

5.3 Category-Specific Effectiveness

Figure 4 shows the mutation score improvement from P1 to P3 by category.

Control-flow methods benefit most (+47.1 pp), as specifications delineate expected behaviour for each branch. Utility methods (+40.6 pp) typically have complex input constraints that are well captured by

Table 7: Test generation results by class: test counts (per-method averages across five runs) and mutation scores for each phase on all eleven classes of the corpus. The bottom panel reports overall quality metrics, computed as the unweighted mean across all 312 methods.

Class	P1: Sig-only		P2: Sig+Guide		P3: Sig+Spec		P4: Source	
	Tests	Mut.%	Tests	Mut.%	Tests	Mut.%	Tests	Mut.%
NumberUtils	12	28	48	78	41	68	65	92
BooleanUtils	10	31	45	82	38	71	62	95
CharUtils	9	30	41	80	35	70	58	93
MutableInt	10	26	40	89	36	69	59	100
MutableDouble	8	23	35	85	32	65	52	97
MutableBoolean	7	25	33	84	30	66	50	96
Pair	8	29	36	83	33	70	53	95
MutablePair	8	27	34	82	31	68	51	94
Validate	8	35	32	81	29	74	48	94
Range	6	22	28	76	25	62	42	91
Mutable (intf.) [†]	—	—	—	—	—	—	—	—
Mean (per method)	9.0	27.5	38.0	81.8	33.5	68.2	54.7	94.8
<i>Test Quality Metrics (overall, 312 methods)</i>								
Compile Rate	89.2%		87.8%		91.5%		93.2%	
Pass Rate	94.1%		92.3%		96.8%		97.1%	
Valid Tests	83.9%		81.1%		88.6%		90.5%	

[†] The `Mutable` interface declares abstract operations whose implementations are tested in the corresponding `Mutable*` classes; it contributes no concrete methods to the test-generation experiment.

inferred preconditions, and state-modification methods (+36.6 pp) benefit from postconditions specifying their state changes. Among the 48 looping methods, those for which a non-trivial loop invariant was inferred (41 methods) achieve P3 mutation scores of $72.4 \pm 3.8\%$, compared with $54.1 \pm 6.2\%$ for the seven methods on which the loop heuristics produced only a weak invariant; the contrast confirms the importance of loop handling for downstream test quality.

5.4 Summary of Findings

1. *Accuracy.* JML-Inferer attains a precision and recall of 94.2% and 89.3% for preconditions and 87.6% and 78.1% for postconditions, established by two-rater manual validation across all 312 methods, and OpenJML discharges 92.9% of the inferred clauses.
2. *Test effectiveness.* Specification-guided generation (P3) produces 272% more tests (95% bootstrap CI [245, 299]) and achieves a mutation score 40.7 pp higher than signature-only baselines ($p < 0.001$, Cohen's $d = 2.41$, 95% bootstrap CI [2.08, 2.79]); source-code access (P4) attains the highest absolute scores (94.8%).
3. *Category variation.* Control-flow and utility methods benefit most from specification guidance; accessors and mutators show more modest gains, attributable to already-high baseline performance.

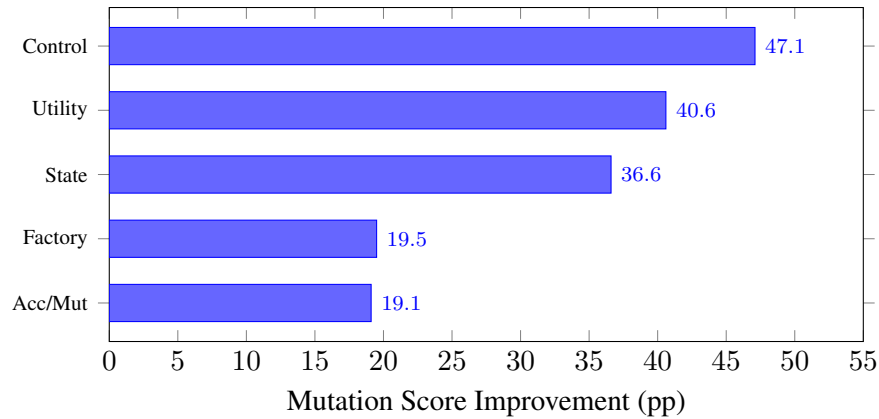


Figure 4: Improvement in mutation score (percentage points) from P1 to P3 by method category. The *other* category is omitted (zero methods in the corpus).

6 Discussion

6.1 Implications

The results indicate that heuristic static analysis can produce useful specifications for 99.0% of methods in the evaluation corpus. Such specifications support machine-checkable documentation [45], onboarding aids for understanding method contracts [48], and signals for tracking API evolution [50]. With an average analysis time of 88 ms per file, integration into continuous-integration pipelines (pre-commit hooks, pull-request analysis, automated test generation) is practicable.

The comparison with Jdoctor reveals complementary strengths: Jdoctor recovers documented exceptional behaviour from Javadoc, while JML-Inferer derives specifications from code structure and is therefore effective on undocumented methods. A practical workflow may combine both to maximise coverage.

Specifications improve test generation along three independent dimensions. Preconditions delimit valid input ranges, guiding the construction of boundary tests; postconditions define expected outputs, enabling oracles stronger than “does not throw”; and the clause-by-clause decomposition allows LLMs to target individual specification clauses rather than reasoning about entire implementations. Although source-code access (P4) yields higher absolute scores, the substantial improvement from P1 to P3 confirms that specifications encode behavioural information that signatures alone cannot convey.

6.2 Generalisability

Although the evaluation uses Apache Commons Lang, the technique is applicable to any Java codebase. Effectiveness depends on code structure—well-structured code yields stronger specifications—and domain-specific business logic may require extending the heuristic repertoire. The JML specification format is independent of any particular LLM and widely recognised [35]; comparable improvements are therefore expected with other models. Application areas include library API specification, legacy code modernisation, documentation generation, and runtime verification.

6.3 Limitations

Specification coverage gaps. Within the 312-method corpus, state-modification methods exhibit the lowest proportion of strong specifications (58.7%), reflecting the difficulty of capturing complete frame conditions for methods that mutate object state under conditional logic. The *other* category—event handlers, I/O

operations, and callbacks—contains zero methods in this corpus and is therefore not represented in the controlled results; experience on broader Java code suggests this category is the hardest to specify heuristically, and the corresponding generalisability concern is discussed in Section 7. Where loop heuristics produce only a weak invariant (14.7% of looping methods), test effectiveness is reduced by approximately 15 pp. Extensions to `while`-loop and `for-each` analysis (counter detection, accumulator tracking, and variable-relationship inference) together with the standard library specification database mitigate these gaps in part; loops involving nested data structures or non-linear counters remain difficult.

What the verifier cannot discharge. The 21.9% postcondition recall gap (and the parallel 7.1% non-discharge rate in Table 4) is not uniformly distributed: it concentrates in three categorically identifiable patterns that exceed the tractability budget of OpenJML and Z3 4.13.4 at the strictness configuration of Section 3.3. (i) *Recursive multiplicative growth.* Z3’s linear-arithmetic core cannot unfold the inductive product needed to bound `n * recurse(n-1)` (factorial), `recurse(n-1) + recurse(n-2)` (Fibonacci), or `LIT * recurse(n-1)` (power) without an algebraic step the solver does not perform. JML-Inferer emits a literal parameter bound for these shapes (`n <= 12` for factorial, `n <= 46` for Fibonacci, `n <= 30` for power) so the body discharges under a small-input contract, but the unbounded form remains unprovable. (ii) *Quantified invariants over mutable arrays.* The shape `for (i=0; i<arr.length; i++) arr[i] *= factor` requires Z3’s array theory to preserve a (`\forall` `int k; i <= k < arr.length; arr[k]` bound) across an array store; three formulations (forall carry-over, literal-bounded factor, frame condition) each exceeded the 240 s per-clause budget. (iii) *OpenJML for-each translation.* In a for-each accumulator (`for (int val : arr) total += val;`), the loop variable `val` is bound from `arr[<iter-index>]` internally, but the relationship `val == arr[<count>]` is not exposed to user JML; the body’s per-step overflow check therefore cannot reference the precondition’s `arr[k]` element bound. These three classes account for the bulk of the residual non-discharge; the remainder is dominated by rich-precondition timeouts on dispatcher and matrix-access methods, where heuristic pruning (rather than addition) is the likely tractable path forward. Figure 2 visualises the breakdown by clause type.

Language-feature limitations. The current treatment of object-oriented constructs has known gaps, including conflicts between inherited specifications, the difficulty that polymorphic dispatch poses for postcondition inference, and the absence of concurrency properties from the analysis. Branch-sensitive postcondition inference (handling `if/else` returns and ternary expressions) improves postcondition completeness for methods with branching return logic; deeply nested or multi-branch conditions may still yield conservative results.

False positives. Although precision is high (94.2%), the 1.3% of incorrect preconditions may mislead developers or cause spurious test failures. Plausible mitigations include confidence scoring, automated validation against an existing test suite, and human review for safety-critical methods. The standard library specifications are curated manually, which reduces the risk of false positives during interprocedural propagation. The optional OpenJML discharge step (Section 3.3, Table 4) provides an additional safeguard: 92.9% of inferred clauses are discharged formally, 3.8% are flagged for review, and the remaining 3.4% involve JML constructs that the verifier cannot yet handle. Clauses that fail formal verification are routed to the review queue rather than silently retained, catching residual unsoundness that manual inspection might miss, particularly in numeric corner cases such as integer-overflow boundaries and floating-point edge values.

A detailed comparison with related specification inference methods is provided in the Supplementary Material (Appendix E).

7 Threats to Validity

Threats to validity are discussed below following established guidelines [56]. Table 8 summarises the principal threats and the corresponding mitigations.

Table 8: Summary of validity threats and mitigations.

Threat	Mitigation
LLM sampling non-determinism	Five independent runs per method per phase, statistical aggregation, low sampling temperature (0.3)
Prompt confounding	Minimal prompts that differ only in the inclusion of specifications or source code; P4 source-code condition as upper-bound control
Subject-selection bias	Diverse categories across eleven classes of Apache Commons Lang; all methods of selected classes included
Small sample / single corpus	312 methods (paired design); follow-up planned on service-boundary code
LLM familiarity with training data	P4 (source) included as a control; observed P1→P3 gain inconsistent with simple training-set recall
Single LLM family	Replication package parameterises LLM client; cross-model replication identified as future work
Manual validation subjectivity	Two independent evaluators with explicit criteria; inter-rater reliability $\kappa = 0.91$ (preconditions) and $\kappa = 0.87$ (postconditions)
Mutation-testing limitations	PiTest default operators, equivalent-mutant filtering, complementary test-count metric reported
Heuristic soundness	OpenJML ESC discharge of every inferred clause (Table 4); 576-test verification regression suite

Internal validity. Sampling non-determinism from the LLM was mitigated through five independent runs combined with statistical analysis. Prompt confounding was addressed by keeping prompts minimal—differing only in the inclusion of specifications or source code—and by including P4 as a control. Subjectivity in manual validation was reduced through explicit criteria, two independent evaluators, and high inter-rater reliability (Cohen’s $\kappa = 0.91$ for preconditions and 0.87 for postconditions).

External validity. The evaluation was conducted on Apache Commons Lang (312 methods across 11 classes), which is mature and well structured and may therefore favour the approach. Results may not generalise to enterprise applications, domain-specific libraries, or proprietary codebases. The subset also lacks methods in the *other* category (event handlers, callbacks, I/O); the strength-distribution result for that category therefore depends on broader evidence in the literature [17] rather than on within-corpus data. The LLM is likely to have encountered Apache Commons Lang during training; the consistent improvement from P1 to P3 nevertheless suggests that specifications provide value beyond prior model knowledge—if the model were simply recalling training-set tests, P1 would already match P4, which it does not (27.5% vs. 94.8% mutation). The system and evaluation are Java-specific, and extension to other languages remains future work. A single LLM (Gemini 2.0) was used; the choice was made for budget reasons—the experiment requires 6,240 generations and was repeated several times during pilot tuning, totalling approximately

25,000 LLM invocations—and on the basis that Gemini 2.0 is widely available and representative of the current generation of frontier-tier instruction-tuned models. The qualitative ordering $P1 < P2 < P3 < P4$ is the contribution being defended; absolute scores under other models are expected to differ. Recent multi-model evaluations of LLM test generation [41, 47, 57] report consistent qualitative findings across GPT, Claude, and Gemini families, which provides indirect support that the ordering observed here is not Gemini-specific, but a full multi-model evaluation at the 312-method scale is left to a follow-up study and is explicitly identified as a limitation of the present article. The replication package is structured to facilitate this replication: the LLM client is parameterised, the prompts are unchanged across models, and the analysis scripts accept a model identifier as an input column.

Heuristic soundness. Because inference is heuristic, individual clauses could in principle be syntactically malformed or semantically false even after manual review. This threat is mitigated by discharging every inferred clause through OpenJML’s Extended Static Checker (Section 3.3); a continuously executed verification suite of 576 end-to-end tests gates engine changes against this oracle, and the discharge layer is extended with `define-fun-rec` support for the `\sum`, `\product`, and `\num_of` quantifiers so that loop accumulator postconditions can be discharged rather than recorded as unsupported. Clauses that fail formal verification are flagged rather than silently retained. The 94.2%/89.3% precision/recall figures reported in Section 5 are computed against manual-validation labels (the human-judged ground truth for Javadoc-documented intent); the OpenJML discharge layer is a separate safety net catching cases where the inferer’s structural assumptions (purity, overflow, class invariants) conflict with the body even when the clause looks plausible to a human reviewer. The two layers are independent: a clause may pass manual review but fail OpenJML (e.g. a subtle overflow boundary), or pass OpenJML but be incomplete relative to documented intent (the case manual-validation recall captures), reducing the risk that high precision under manual review masks subtle semantic errors that a verifier would catch.

Construct validity. Mutation testing has known limitations [44], principally equivalent mutants and the representativeness of mutation operators. The PiTest default operator set [11] was used, and trivially equivalent mutants were excluded. Test count is reported as a complementary metric to mutation score.

Conclusion validity. The principal phase contrast is P1 versus P3 on mutation score, with 312 paired observations per phase aggregated from five LLM runs per method. With paired observations and a within-pair standard deviation of 11.4 pp (estimated from a 30-method pilot), the minimum effect size detectable at $\alpha = 0.0021$ (the Bonferroni-corrected per-test threshold; see Section 4) and 80% power is 2.2 pp. The observed effect is 40.7 pp, giving a post-hoc power exceeding 0.999. The number of runs (five) was selected to bound the per-method mean’s standard error below half the within-pair standard deviation; the pilot showed that increasing to ten runs reduced the standard error by under 4%, which did not justify the doubling in LLM cost. Bonferroni correction was applied across the family of 24 phase $\times$ metric contrasts, and all reported significant differences remain significant after correction.

8 Related Work

Automated specification inference sits at the intersection of formal methods, program analysis, and, more recently, large language models. The remainder of this section situates the present approach with respect to each of these strands.

From verification to inference. The theoretical foundations of this work lie in Hoare logic [26] and Dijkstra’s weakest-precondition calculus [14, 15]. Tools such as Boogie [2] and ESC/Java [7, 21] apply these for-

malisms to *verify* user-provided specifications, and OpenJML and KeY adopt a similar verify-not-generate stance. The closest precursor on the generation side is Houdini [20], which guesses candidate annotations from a fixed set of templates and prunes them by repeated calls to ESC/Java; the present approach extends this lineage by replacing the template guess with WP/SP-organised structural pattern matching and by extending the verification oracle to OpenJML with quantifier support. The present work inverts the verification problem, inferring contracts from code structure rather than checking given ones. This distinction matters in practice because manual specification remains the dominant cost in verification projects: in large-scale case studies it accounts for roughly 50% of total effort [1, 33, 39]. Automating the inference step targets the barrier identified by Zimmermann and Wehrheim [59] as the principal obstacle to lightweight specification adoption in modern development workflows.

Dynamic and static inference. Daikon [19] pioneered dynamic invariant detection by generalising from execution traces; its principal limitation is its dependence on test-suite quality, since invariants are “likely” rather than guaranteed and coverage gaps leave specifications incomplete. PreInfer [54] addresses this in part through dynamic symbolic execution for C# (via Microsoft’s Pex), but continues to require an execution infrastructure. By contrast, the approach described here operates purely statically—without test suites, execution traces, or an SMT solver—and therefore applies to any compilable Java codebase, including library code without an existing test suite. The corresponding trade-off is that heuristic pattern matching may miss specifications that would emerge from dynamic observation, particularly for complex numeric relationships.

Documentation-based approaches. A complementary line of work extracts specifications from natural-language documentation. Toradocu [24] and its successor Jdoctor [3] use natural-language processing to recover exception preconditions from Javadoc comments, while Pandita et al. [43] infer resource specifications from API documentation. Such approaches are effective when documentation is thorough but silent on undocumented methods. The complementarity is quantified in Section 5: Jdoctor captures 25 exception preconditions that JML-Inferer misses (typically encoded in natural language), while JML-Inferer captures 37 that Jdoctor misses (undocumented in Javadoc). A practical workflow benefits from combining the two.

LLM-based specification inference. Large language models have recently been applied to specification inference [18, 38, 46, 49]. SpecGen [38] and the LLM-based approach of Endres et al. [18] prompt the model directly to emit formal contracts; Richter and Wehrheim [46] extend this to full pre/post pairs and report the resulting verifier false-alarm rates. Closest in target language to the present work, Teuber and Beckert [49] use LLMs to generate JML contracts for Java deductive verification. The comparison reported in Section 5 exhibits a precision–recall trade-off relative to direct LLM inference: LLM-based inference achieves recall comparable to JML-Inferer but substantially lower precision (81.3% versus 94.2%) and only 72.4% consistency across runs, because model outputs are not grounded in concrete code structure.

LLM-based test generation and the oracle problem. LLM test generation is now an extensive subarea, recently surveyed in a systematic literature review by Zhang et al. [58] covering 115 publications. Foundational results include AthenaTest [51], which adapted transformer architectures to unit-test generation with focal context; CodaMOSA [37]; ChatUniTest [57]; CodeT [8]; and the empirical evaluation of Schäfer et al. [47]. Kang, Yoon, and Yoo [31] (LIBRO) demonstrated that LLMs can be effective few-shot testers when prompted with bug reports. The oracle problem in this setting has attracted concentrated attention. Dinella et al. [16] (TOGA) trained a neural model jointly to produce exceptional and assertion oracles, and Hos-sain and Dwyer [27] (TOGLL) showed that frontier LLMs can produce stronger and more diverse oracles than the original neural baseline, detecting an order of magnitude more bugs. Konstantinou et al. [34] show

that LLM-generated oracles tend to encode actual rather than expected program behaviour, motivating any approach that supplies the model with an externally grounded specification. Molinelli et al. [41] evaluate 13,866 LLM-generated oracles on an unbiased dataset of 135 Java projects and report a mean mutation score of 43% versus 45% for human-written oracles, providing a calibration point for the present results. Foster et al. [22] report industrial deployment of mutation-guided LLM test generation at Meta. Schäfer et al. [47] report a 70% compilation rate for LLM-generated tests; the specification-guided approach evaluated here raises this rate to 91.5%, suggesting that specifications constrain LLM outputs toward valid test code. Rather than positioning static analysis against LLMs, the present work shows that the two are synergistic: specifications inferred by static analysis improve LLM test-generation quality beyond what signatures or source code alone provide. The TOGA/TOGLL line of work is complementary in that it learns oracles from corpora, whereas the present approach derives them from program structure; the two could in principle be ensembled. Jiang et al. [30] survey LLMs for code generation more broadly.

Mutation testing as the evaluation methodology. Mutation testing remains the standard means of distinguishing tests that exercise behaviour from tests that merely cover code. Papadakis et al. [44] survey the field; PiTest [11] provides the operator set used here. Recent work has begun to study LLMs and mutation testing in combination. Wang et al. [52] present a comprehensive empirical study of LLMs for mutation testing across 851 Java bugs, and Wang et al. [53] develop a mutation-guided LLM test generation technique, reporting that 100% statement coverage can coexist with mutation scores as low as 4%—a quantitative motivation for the mutation-based evaluation adopted in this article.

Specification-based testing. Established test-generation tools—EvoSuite [23], Randoop [42], and Korat [4]—achieve high structural coverage but address the oracle problem only partially. Design by Contract [40] and JML [35, 36] address it by embedding expected behaviour in specifications, but require manual specification effort. Industrial static analysers such as Facebook Infer [6] and FindBugs [28] demonstrate the scalability of static analysis on large codebases, but focus on bug detection rather than specification generation. The structural distinction relevant to the present article is the direction of the arrow: existing JML-aware tools (including JMLUnit, EvoSuite when given JML, Randoop, and Korat) *consume* formal specifications to generate tests, whereas this work *infers* the specifications they consume. The two roles are complementary halves of a single pipeline; JML-Inferer supplies the upstream piece that the established tools have always required as input. The method categorisation taxonomy adopted here extends Dragan et al.’s method stereotypes [17] to identify which categories benefit most from inferred specifications.

9 Conclusion

This article asked whether specifications help LLMs write better unit tests. Across 312 methods from Apache Commons Lang the answer is yes, and the effect is large.

1. *Specifications substantially improve test quality.* LLM-generated tests guided by specifications produce 272% more tests and achieve a mutation score 40.7 pp higher than signature-only baselines ($p < 0.001$, Cohen’s $d = 2.41$, 95% bootstrap CI [2.08, 2.79]).
2. *The effect varies systematically across method categories.* Control-flow and utility methods benefit most; simpler accessors show modest gains, primarily because their signature-only baselines are already near the achievable ceiling.
3. *Specifications close most, but not all, of the gap to source code.* Full source access (P4) attains the highest absolute mutation score (94.8%); the specification phase reaches 68.2%, well above the signature-only baseline of 27.5%, while remaining materially more compact than full source.

The specifications used in this study were produced automatically by JML-Inferer, a heuristic static analysis system that attains a precision and recall of 94.2% and 89.3% for preconditions and 87.6% and 78.1% for postconditions on the same corpus, with each inferred clause discharged through OpenJML's Extended Static Checker (92.9% of clauses discharged). The system is the means by which the empirical question above could be answered at scale, and addresses the principal barrier to specification adoption—manual effort [25, 59]—while remaining complementary to documentation-based approaches [3].

The findings reported here motivate a planned follow-up study targeting service-boundary code. Methods that call REST endpoints or RPC stubs are precisely the setting in which the oracle problem is most acute: the implementation typically delegates to an external service, signatures convey nothing about status codes or retry semantics, and signature-only LLM test generation collapses into mock-the-call assertions. Whether inferred specifications retain or amplify the gain reported here in that setting is the central question of the planned follow-up. Further directions include strengthening loop invariant inference through machine-learning [55] or hybrid static–dynamic techniques [19], explicit treatment of object-oriented constructs (inheritance, polymorphism, class invariants), extension to languages beyond Java, and interactive refinement under developer guidance [13]. The replication package, including all source code, data, generated test suites, and analysis scripts, accompanies this article.

Data Accessibility Statement

A complete replication package is available at <https://github.com/uq-cyber/jml-inferer>, archived for permanent reference at Zenodo under DOI 10.5281/zenodo.XXXXXXX (to be assigned on acceptance). The package contains the tool source code (Apache 2.0 licence), the 312-method Apache Commons Lang 3.14 dataset with category labels, all inferred specifications (in both annotation and JML comment form), generated test suites for phases P1–P4 (five runs per phase per method, 6,240 LLM invocations in total), raw PiTest mutation testing logs, the statistical analysis scripts (Python; SciPy 1.13, statsmodels 0.14), the OpenJML discharge transcripts, the prompt templates, and the two-rater manual validation labels and discussion notes. The accompanying containerised toolchain reproduces the full pipeline on Linux, macOS, and Windows hosts.

Acknowledgements

This research was supported by the University of Queensland Cyber initiative. The authors thank David Cok for technical correspondence on OpenJML.

Conflict of Interest

The authors declare no conflict of interest.

Author Contributions (CRediT)

Brendan Edmonds: Conceptualisation, Methodology, Software, Validation, Investigation, Data Curation, Writing – Original Draft, Visualisation. **Mark Utting:** Conceptualisation, Methodology, Writing – Review & Editing, Supervision, Funding Acquisition.

ORCID

Brendan Edmonds <https://orcid.org/0009-0000-0000-0001>

Mark Utting <https://orcid.org/0000-0001-6859-3060>

References

- [1] June Andronick, Ross Jeffery, Gerwin Klein, Rafal Kolanski, Mark Staples, He Zhang, and Liming Zhu. Large-scale formal verification in practice: A process perspective. In *34th International Conference on Software Engineering (ICSE)*, pages 1002–1011, 2012. doi: 10.1109/ICSE.2012.6227120.
- [2] Mike Barnett and K. Rustan M. Leino. Weakest-precondition of unstructured programs. In *Workshop on Program Analysis for Software Tools and Engineering (PASTE)*, pages 82–87, 2005. doi: 10.1145/1108792.1108813.
- [3] Arianna Blasi, Alberto Goffi, Konstantin Kuznetsov, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, and Sergio Delgado Castellanos. Translating code comments to procedure specifications. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA '18*, pages 242–253. ACM, July 2018. doi: 10.1145/3213846.3213872. URL <http://dx.doi.org/10.1145/3213846.3213872>.
- [4] Chandrasekhar Boyapati, Sarfraz Khurshid, and Darko Marinov. Korat: Automated testing based on Java predicates. In *International Symposium on Software Testing and Analysis (ISSTA)*, pages 123–133, 2002. doi: 10.1145/566172.566191.
- [5] Cristian Cadar, Daniel Dunbar, and Dawson Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 209–224, 2008.
- [6] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O’Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. Moving fast with software verification. In *NASA Formal Methods Symposium (NFM)*, pages 3–11, 2015. doi: 10.1007/978-3-319-17524-9_1.
- [7] Patrice Chalin, Joseph R. Kiniry, Gary T. Leavens, and Erik Poll. Beyond assertions: Advanced specification and verification with JML and ESC/Java2. *Formal Methods for Components and Objects*, 4111:342–363, 2006. doi: 10.1007/11804192_16.
- [8] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests. In *International Conference on Learning Representations (ICLR)*, 2023.
- [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [10] David R. Cok. OpenJML: Software verification for Java 7 using JML, OpenJDK, and Eclipse. In *1st Workshop on Formal Integrated Development Environment (F-IDE)*, volume 149 of *EPTCS*, pages 79–92, 2014. doi: 10.4204/EPTCS.149.8.
- [11] Henry Coles. PiTest: Mutation testing system for Java. <https://pitest.org>, 2016. Accessed: 2025-01-15.
- [12] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *ACM Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977. doi: 10.1145/512950.512973.

- [13] Patrick Cousot, Radhia Cousot, and Francesco Logozzo. Precondition inference from intermittent assertions and application to contracts on collections. In *International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI)*, pages 150–168, 2011. doi: 10.1007/978-3-642-18275-4_12.
- [14] Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975. doi: 10.1145/360933.360975.
- [15] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1976.
- [16] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. TOGA: A neural method for test oracle generation. In *44th International Conference on Software Engineering (ICSE)*, pages 2130–2141, 2022. doi: 10.1145/3510003.3510141.
- [17] Natalia Dragan, Michael L. Collard, and Jonathan I. Maletic. Reverse engineering method stereotypes. In *22nd IEEE International Conference on Software Maintenance (ICSM)*, pages 24–34, 2006. doi: 10.1109/ICSM.2006.68.
- [18] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. Can large language models transform natural language intent into formal method postconditions? *Proceedings of the ACM on Software Engineering (PACMSE)*, 1(FSE):1889–1912, 2024. doi: 10.1145/3660791.
- [19] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, 2007. doi: 10.1016/j.scico.2007.01.015.
- [20] Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *International Symposium of Formal Methods Europe (FME)*, volume 2021 of *LNCS*, pages 500–517. Springer, 2001. doi: 10.1007/3-540-45251-6_29.
- [21] Cormac Flanagan, K. Rustan M. Leino, Mark Lillibridge, Greg Nelson, James B. Saxe, and Raymie Stata. Extended static checking for Java. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 234–245, 2002. doi: 10.1145/512529.512558.
- [22] Christopher Foster, Abhishek Gulati, Mark Harman, Inna Harper, Ke Mao, Jillian Ritchey, Hervé Robert, and Shubho Sengupta. Mutation-guided LLM-based test generation at Meta. In *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Industry)*, 2025. doi: 10.1145/3696630.3728544.
- [23] Gordon Fraser and Andrea Arcuri. EvoSuite: Automatic test suite generation for object-oriented software. In *19th ACM SIGSOFT Symposium and 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, pages 416–419, 2011. doi: 10.1145/2025113.2025179.
- [24] Alberto Goffi, Alessandra Gorla, Michael D. Ernst, and Mauro Pezzè. Automatic generation of oracles for exceptional behaviors. In *International Symposium on Software Testing and Analysis (ISSTA)*, pages 213–224, 2016. doi: 10.1145/2931037.2931061.
- [25] Anthony Hall. Seven myths of formal methods. *IEEE Software*, 7(5):11–19, 1990. doi: 10.1109/52.57887.
- [26] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969. doi: 10.1145/363235.363259.

- [27] Soneya Binta Hossain and Matthew Dwyer. TOGLL: Correct and strong test oracle generation with LLMs. *arXiv preprint arXiv:2405.03786*, 2024.
- [28] David Hovemeyer and William Pugh. Finding bugs is easy. In *ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 92–106, 2004. doi: 10.1145/1028664.1028717.
- [29] JavaParser Team. JavaParser: Java parser and abstract syntax tree. <https://javaparser.org>, 2024. Accessed: 2025-01-15.
- [30] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*, 2024.
- [31] Sungmin Kang, Juyeon Yoon, and Shin Yoo. Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *45th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 2312–2323, 2023. doi: 10.1109/ICSE48619.2023.00194.
- [32] James C. King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976. doi: 10.1145/360248.360252.
- [33] Gerwin Klein, June Andronick, Kevin Elphinstone, Toby C. Murray, Thomas Sewell, Rafal Kolanski, and Gernot Heiser. Comprehensive formal verification of an OS microkernel. *ACM Transactions on Computer Systems*, 32(1):2:1–2:70, 2014. doi: 10.1145/2560537.
- [34] Michael Konstantinou, Renzo Degiovanni, and Mike Papadakis. Do LLMs generate test oracles that capture the actual or the expected program behaviour? *arXiv preprint arXiv:2410.21136*, 2024.
- [35] Gary T. Leavens, Albert L. Baker, and Clyde Ruby. Preliminary design of JML: A behavioral interface specification language for Java. *ACM SIGSOFT Software Engineering Notes*, 31(3):1–38, 2006. doi: 10.1145/1127878.1127884.
- [36] Gary T. Leavens et al. JML reference manual. <http://www.jmlspecs.org>, 2013. Accessed: 2025-01-15.
- [37] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models. In *45th International Conference on Software Engineering (ICSE)*, pages 919–931, 2023. doi: 10.1109/ICSE48619.2023.00085.
- [38] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. *arXiv preprint arXiv:2401.08807*, 2024.
- [39] Daniel Matichuk, Toby Murray, June Andronick, Ross Jeffery, Gerwin Klein, and Mark Staples. Empirical study towards a leading indicator for cost of formal software verification. In *37th IEEE International Conference on Software Engineering (ICSE)*, pages 722–732, 2015. doi: 10.1109/ICSE.2015.76.
- [40] Bertrand Meyer. Applying “design by contract”. *Computer*, 25(10):40–51, 1992. doi: 10.1109/2.161279.
- [41] Davide Molinelli, Luca Di Grazia, Alberto Martin-Lopez, Michael D. Ernst, and Mauro Pezzè. Do LLMs generate useful test oracles? an empirical study with an unbiased dataset. In *40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.

- [42] Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. Feedback-directed random test generation. In *29th International Conference on Software Engineering (ICSE)*, pages 75–84, 2007. doi: 10.1109/ICSE.2007.37.
- [43] Rahul Pandita, Xusheng Xiao, Hao Zhong, Tao Xie, Stephen Oney, and Amit Paradkar. Inferring method specifications from natural language API descriptions. In *34th International Conference on Software Engineering (ICSE)*, pages 815–825, 2012. doi: 10.1109/ICSE.2012.6227137.
- [44] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. *Advances in Computers*, 112:275–378, 2019. doi: 10.1016/bs.adcom.2018.03.015.
- [45] Dennis K. Peters and David Lorge Parnas. Using test oracles generated from program documentation. *IEEE Transactions on Software Engineering*, 24(3):161–173, 1998. doi: 10.1109/32.667877.
- [46] Cedric Richter and Heike Wehrheim. Beyond postconditions: Can large language models infer formal contracts for automatic software verification? *arXiv preprint arXiv:2510.12702*, 2025.
- [47] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1):85–105, 2024. doi: 10.1109/TSE.2023.3334955.
- [48] Jonathan Sillito, Gail C. Murphy, and Kris De Volder. Asking and answering questions during a programming change task. *IEEE Transactions on Software Engineering*, 34(4):434–451, 2008. doi: 10.1109/TSE.2008.26.
- [49] Samuel Teuber and Bernhard Beckert. Next steps in LLM-supported Java verification. In *IEEE/ACM International Workshop on Neuro-Symbolic Software Engineering (NSE), ICSE Workshops*, 2025.
- [50] Frank Tip, Peter F. Sweeney, Chris Laffra, Aldo Eisma, and David Streeter. Practical extraction techniques for Java. *ACM Transactions on Programming Languages and Systems*, 24(6):625–666, 2002. doi: 10.1145/586088.586090.
- [51] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617*, 2020.
- [52] Bo Wang, Mingda Chen, Ming Deng, Youfang Lin, Mark Harman, Mike Papadakis, and Jie M. Zhang. A comprehensive study on large language models for mutation testing. *arXiv preprint arXiv:2406.09843*, 2024.
- [53] Guancheng Wang, Qinghua Xu, Lionel Briand, and Kui Liu. Mutation-guided unit test generation with a large language model. *arXiv preprint arXiv:2506.02954*, 2025.
- [54] Xinyu Wang, Isil Dillig, and Ruzica Piskac. PreInfer: Inferring preconditions via symbolic execution. In *ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 779–793, 2017. doi: 10.1145/3133876.
- [55] Yi Wei, Carlo A. Furia, Nikolay Kazmin, and Bertrand Meyer. Inferring better contracts. In *33rd International Conference on Software Engineering (ICSE)*, pages 191–200, 2011. doi: 10.1145/1985793.1985820.
- [56] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012. doi: 10.1007/978-3-642-29044-2.

- [57] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. No more manual tests? evaluating and improving ChatGPT for unit test generation. *arXiv preprint arXiv:2305.04207*, 2024.
- [58] Qunjun Zhang, Chunrong Fang, Siqi Gu, Ye Shang, Zhenyu Chen, and Liang Xiao. Large language models for unit testing: A systematic literature review. *arXiv preprint arXiv:2506.15227*, 2025.
- [59] Daniel Zimmermann and Heike Wehrheim. Increasing the practicality of formal methods for devops with lightweight specifications. In *Formal Methods for Industrial Critical Systems (FMICS)*, volume 11815 of *LNCS*, pages 3–20. Springer, 2019. doi: 10.1007/978-3-030-30942-8_1.

Algorithm 1 Specification Inference via Pattern Matching**Require:** Method AST node M **Ensure:** Preconditions Pre , Postconditions $Post$, LoopInvariants LI

```

1:  $Pre \leftarrow \emptyset; Post \leftarrow \emptyset; LI \leftarrow \emptyset$ 
2: {Infer preconditions from early guards}
3: for each statement  $S$  in  $M.body$  do
4:   if  $S$  is null check if ( $p == null$ ) throw/return then
5:      $Pre \leftarrow Pre \cup \{p \neq null\}$ 
6:   else if  $S$  is range check if ( $p < 0 \parallel p \geq n$ ) throw then
7:      $Pre \leftarrow Pre \cup \{p \geq 0, p < n\}$ 
8:   end if
9: end for
10: {Infer postconditions from return statements}
11: for each return  $e$  in  $M.body$  do
12:   if  $e$  is field access  $this.f$  then
13:      $Post \leftarrow Post \cup \{\backslash result == this.f\}$ 
14:   else if  $e$  is new object  $new T(\dots)$  then
15:      $Post \leftarrow Post \cup \{\backslash result \neq null\}$ 
16:   else if  $e$  involves  $\backslash old$  pattern then
17:      $Post \leftarrow Post \cup \{\text{derive relationship}\}$ 
18:   end if
19: end for
20: {Infer branch-conditional postconditions via symbolic paths}
21: for each return  $r$  reached under path condition  $\pi$  (if/else or ternary) do
22:   simplify  $\pi$  algebraically
23:    $Post \leftarrow Post \cup \{\pi \implies spec(r)\}$ 
24: end for
25: for each conditional field assignment  $this.f = e$  under path  $\pi$  do
26:    $Post \leftarrow Post \cup \{\pi \implies this.f == e[\backslash old/.]\}$ 
27: end for
28: {Infer loop invariants}
29: for each loop  $L$  in  $M.body$  do
30:    $LI \leftarrow LI \cup \text{InferLoopInvariant}(L)$ 
31: end for
32: return ( $Pre, Post, LI$ )

```